

One package or two?

AUTHOR

Giovanni

As you know, the `plotGraph` function (based on `igraph::tkplot`) will no longer be available because it will be removed in the next version of `igraph`.

I have developed (together with Google Gemini) a replacement function called (tentatively) `graphLayout`, based on the **shine** and **visNetwork** packages.

But I was wondering: instead of including `graphLayout` in the `ggm` package along with two additional packages, wouldn't it be better to mark `plotGraph` as "deprecated" and create a new, lightweight package that contains only `graphLayout` and a few other auxiliary functions?

I tried asking Gemini for his opinion beforehand so I could then run it by you.

Option A

Create a new, separate package (e.g., **ggmLayout**)

The Benefits:

1. `ggm` remains extremely lightweight: The main package maintains a streamlined structure, focused solely on graph algebra, matrices, and maximum likelihood estimation. Users who rely on `ggm` solely for calculations in automated scripts or on servers don't have to carry the entire **shiny** infrastructure with them. Of course the functions to generate adjacency variables for mixed variables, like `grMAT` and `makeMG` plus all your functions like `RG`, `SG`, `AG`, `Max`, `mSep` etc. will remain in our package. Moreover `plotGraph` may be limited to the argument `'tcltk = TRUE'`
2. Independent development cycles: If you want to update the graphical interface, add new visual options, or fix JavaScript bugs in the future, you can release updates on CRAN for the graphics package without having to modify or resubmit the mathematical code of `ggm`.
3. Clear identity: It makes it obvious to users that this is a modern, interactive extension of the classic framework.

Disadvantage:

- Fragmentation: Users who have been accustomed for years to running `library(ggm)` and then `plotGraph()` will be taken aback to see the function disappear and will have to learn how to install and load a second package.

Option B

Keep it in `ggm` under `Suggests` (the current approach)

Since we've structured the function using the `requireNamespace()` check, the two additional packages (**shiny** and **visNetwork**) are not hard dependencies (`Imports` or `Depends`).

- This means that a user can install and run 99% of `ggm` without having **shiny** installed.

- Only when they actually try to call `graphLayout()` does the package detect the missing dependency and tell the user what to do. Many major packages on `CRAN` use this strategy for secondary graphics functions.

Advice

If you decide that separation is the cleanest way to avoid “polluting” the legacy code of **ggm**, the standard procedure on `CRAN` calls for a gradual transition to avoid inconveniencing users:

1. In version 2.5.3 (or later) of **ggm**: Keep the old `plotGraph` but add a warning at the beginning using the `.Deprecated()` function. For example:

```
plotGraph <- function(...) {  
  .Deprecated(msg = "plotGraph (based on tkplot) is deprecated and will be removed. \n  
Please use the new 'ggmLayout' package for interactive plots.")  
  
  # ... old code `tkplot` until igraph removes the function ...  
}
```

2. Create the new package (e.g., **ggmLayout**): This package will include the following entries in its `DESCRIPTION` file: `Depends: ggm` (or `Imports: ggm`), **shiny**, and **visNetwork**.

It will contain the new `graphLayout` directly within the new package, keeping the same syntax!).

This way, anyone who wants modern interactive graphs will simply do the following:

```
library(ggmLayout)  
plotGraph(ag) # And it will use the Shiny + visNetwork engine that we wrote!
```